

Graphical Abstract

A 3D Sampling-based Velocity Obstacles Algorithm for Heterogeneous Robotic Vehicles

Francisco Pires da Silva, Tiago Marques Claro, Daniel Filipe Campos, Andry Maykol Pinto

Highlights

A 3D Sampling-based Velocity Obstacles Algorithm for Heterogeneous Robotic Vehicles

Francisco Pires da Silva, Tiago Marques Claro, Daniel Filipe Campos, Andry Maykol Pinto

- Real-time Obstacle Avoidance in dynamic and uncertain environments.
- 3D spatial representation leveraging 6 Degrees-of-Freedom kinematics.
- Validated in simulated and real-world environments.

A 3D Sampling-based Velocity Obstacles Algorithm for Heterogeneous Robotic Vehicles

Francisco Pires da Silva^{a,b}, Tiago Marques Claro^{a,b}, Daniel Filipe Campos^{a,b}, Andry Maykol Pinto^{a,b}

^a*Department of Electrical and Computer Engineering, Faculty of Engineering, University of Porto, Rua Dr. Roberto Frias, Porto, 4200-465, Porto, Portugal*

^b*INESC TEC, Rua Dr. Roberto Frias, Porto, 4200-465, Porto, Portugal*

Abstract

Ensuring the safe navigation of heterogeneous uncrewed robotic vehicles requires obstacle avoidance methods that address both static and dynamic obstacles in three-dimensional spaces, calculating maneuvers that prevent collisions.

This work proposes and validates the Sampling-based 3D Velocity Obstacles (S3VO), a three-dimensional extension of the Velocity Obstacles approach for heterogeneous vehicles operating in dynamic environments. S3VO introduces a velocity space sampling framework generated from six degrees of freedom (6DOF) kinematic constraints, providing uniform adaptability across heterogeneous robotic platforms, and a multicriteria heuristic function that considers both obstacle proximity and adherence to navigation commands, through heading and magnitude heuristics.

The methodology was validated for a simulated multirotor UAV and compared to state of the art Velocity Obstacle methods, demonstrating the algorithm's ability to maintain safety and reach the goal in 100% of randomly generated scenarios. Further validations were conducted for an holonomic USV and a differential drive USV, first in simulation and subsequently through real-world trials with the differential drive NEST USV at Barragem de Alqueva, Portugal.

Keywords:

Obstacle Avoidance, Velocity Obstacle, Autonomous Navigation

rephrase. combine both

rephrase.

Extensive experiments were made using simulated and real robots (UAV and USV). Results demonstrate ... entrar com quantitative KPI

1. Introduction

Accidents involving surface vehicles in inspections and maintenance of offshore wind farms account for 25% of accidents in these operations [1]. As such, projects such as TALOS¹ and AEROSUB² are demonstrating uncrewed robotic alternatives [2][3][4]. Safe navigation of these robots becomes crucial to make this approach possible. Obstacle avoidance in offshore maritime scenarios has to address complex environments, including both static and dynamic obstacles and navigation in aerial, surface, and underwater domains.

The presented approach, S3VO (Sampling-based 3D Velocity Obstacles), applies the VO algorithm [5] in three dimensions, allowing not only surface vehicles to navigate through three-dimensional dynamic environments, but also supporting full three-dimensional navigation for aerial and underwater platforms, while ensuring consistency across these heterogeneous vehicles.

define consistency

S3VO proposes two distinctive features: a velocity space sampling framework generated from kinematic constraints, making it easily adapted to address different vehicles, and a multicriteria heuristic function to evaluate risk of avoidance maneuvers and to ensure that the algorithm can be integrated into a motion-planning pipeline while respecting the commands specified by the higher-level layers.

This work's contributions are: the following

- A three-dimensional extension of the Velocity Obstacles algorithm (S3VO), enabling navigation in 3D dynamic environments for vehicles with up to six degrees of freedom (6DoF). more innovation features that are unique for this method
- S3VO was implemented for a simulated multirotor UAV and compared with Optimal Reciprocal Collision Avoidance (ORCA). The results highlight some problems of existing approaches on dealing with cluttered environments. S3VO was able to reach the goal with no collisions in 100 out of 100 randomly generated scenarios. Extensive simulation experiments demonstrated...
- S3VO was implemented and evaluated in simulation for two different surface vehicles: a holonomic vehicle and a differential-drive vehicle, and compared with Artificial Potential Fields. Real-world experiments using real USV showed...

¹<https://talosproject.eu/about/>, accessed on June 25, 2025

²<https://www.enlit.world/renewable-energy/wind/project-aerosub-to-develop-robots-for-offshore-wind-om/>, accessed on June 25, 2025

iments with a differential-drive USV in Barragem de Alqueva, Portugal, demonstrate the approach’s capability to handle real-world uncertainties and dynamics, enabling collision-free aquatic navigation.

This paper is divided into 4 sections. Section 2 contains an overview of related work on obstacle avoidance and Velocity Obstacles, and Section 3 presents the proposed algorithm, S3VO. Section 4 analyzes the results and deployment on different simulated and real vehicles. Section 5 presents the main conclusions on the proposed algorithm.

2. Related Work

2.1. Motion Planning and Obstacle Avoidance

Motion planning and obstacle avoidance is a very broad field of research, with applications from robotics to manufacturing, medical, security systems [6], fleet coordination and IoT [7]. Extensive research has been conducted in this field, resulting in a diverse set of approaches. This section proceeds with a description of some of the most widely used and efficient motion planning ideas and algorithms for obstacle avoidance of mobile robots in dynamic environments.

Artificial Potential Fields (APF) [8] is a collision avoidance method that uses virtual forces created by the environment: obstacles create repulsive forces, the goal creates attractive forces, and the resulting potential field is the sum of all attractive and repulsive potentials. The optimal control is then given according to the negative gradient of the APF, $-\nabla U(t)$, by using a controller proportional to this force, for example. This method is simple, fast, and can easily be adapted to a variety of controllers, but is prone to getting stuck in a local minimum of the artificial force field. More recent developments focus on adapting APF for dynamic environments and overcoming the local minimum challenges, using genetic or optimization algorithms [9].

The Dynamic Window Approach (DWA) [10] generates safe velocity commands by searching a reduced space of translational and rotational velocities (v, ω) that are dynamically reachable and allow the robot to stop before hitting obstacles. Candidate velocities are evaluated with an objective function combining target heading, clearance, and forward speed. Later extensions improved DWA for dynamic settings, e.g., Missura and Bennewitz [11], who predict collisions with moving polygonal obstacles. Model Predictive Control (MPC) [12] optimizes control inputs over a finite horizon by solving

constrained optimization problems based on a system model in state-space form. MPC is widely applied to path planning in dynamic environments due to its ability to incorporate dynamics, safety, and traffic-rule constraints. For example, Liu et al. [13] propose an MPC-based unified path planner for autonomous driving, modeling collision avoidance and vehicle dynamics. The key challenges remaining are the high computational cost and reliance on accurate system models. More recently, Kim et al. (2024) [14] explore Imitation Learning (IL) of a Nonlinear Model Predictive Control (NMPC), to preserve NMPC’s collision avoidance but enabling real-time feasibility and Li et al. (2020) [22] explored specialized constraints for maritime environments, such as COLREGs-constrained adaptive fuzzy control.

Elastic Band methods [15] deform a global path into a shorter, smoother collision-free trajectory that adapts online as obstacles appear. The Timed Elastic Band (TEB) approach [16] extends this idea by incorporating temporal information, jointly optimizing robot poses and time intervals under velocity and acceleration constraints. This enables trade-offs between path length and traversal time while enforcing non-holonomic kinematics. To address moving agents, Nguyen et al. [17] propose Proactive TEB (PTEB), combining TEB with Hybrid Reciprocal Velocity Obstacles [27].

Reinforcement Learning (RL) approaches have been increasingly more popular for obstacle avoidance [18] and other path planning tasks [19], like docking of autonomous vehicles [20] or autonomous landing [21]. Using RL for path planning and obstacle avoidance brings adaptability without explicit behavior programming, which is frequently difficult due to the diverse situations that the robot may encounter. Even for humans, choosing the right avoidance maneuver can be challenging, and RL provides total independence from labeling but requires effective reward functions. Other disadvantages are the exploration risks in the learning step, gaps between real-world and simulated environments, and, especially for Deep Reinforcement Learning, lack of interpretability, making it difficult to ensure safety requirements are being met [18].

2.2. Velocity Obstacles

Velocity Obstacles (VO) proposed by Fiorini et al. [23] proposes modeling a velocity space representation merging velocity and position information, to evaluate future collisions. van den Berg et al. propose Reciprocal VO (RVO) [24] to expand to shared responsibility in multi-agent scenarios, and ORCA [25] proposes using linear programming to calculate the closest avoidance

maneuver. Snape et al. [27] introduced Hybrid RVO (HRVO) to deal with deadlock scenarios and introduced uncertainty parameters from Kalman Filter estimation into the velocity space representation. Towards generalizing for non-holonomic vehicles, Wilkie et al. [26] considered non-holonomic constraints in the control space of car-like robots, and Alonso-Mora et al. (2013) [28] differential-drive robot kinematics by inflating the obstacle configuration, considering the error when tracking holonomic equivalents. Towards addressing uncertain environments, Kluge and Prassler (2006) [29] include a probabilistic extension, changing feasible/unfeasible to a probability function. Campos et al. (2019) propose Adaptive VO Algorithm (AVOA) [30], an extension for USVs using Particle Swarm Optimization to find an avoidance maneuver with a multicriteria heuristic using progress to goal and closeness to VO.

Another branch of approaches addresses the fusion of velocity obstacles with deep reinforcement learning. In DRL-VO [31], VO is not the planner but a risk-evaluator inside DRL’s reward function, shaping the agent’s behavior toward COLREGs-compliant and safe actions. Yang et al. (2024) [33] combine DRL with VO, introducing a Navigation Impact Factor based on fuzzy logic to simulate human behavior in navigation tasks. Similarly taking advantage of the fast action pruning provided by VO, Bonanni et al. (2025) [32] propose fusing VO with Monte Carlo Tree Search (MCTS), improving the efficiency of MCTS path planning.

2.3. 3D Velocity Obstacles

Battisti and Muradore (2020) [34] present a solution for expanding ORCA to 3D vehicles. The authors propose using *dummy goals* to tackle deadlocks that arise when the three-dimensional expansion of ORCA does not work as expected. One such deadlock, called *Agent Alignment*, occurs when the algorithm is unable to choose a side to pass the obstacle, causing the agent to remain on the same line. The second arises when moving through closely spaced agents, where the superposition of the ORCA half-planes causes the agent to move backward.

Jenie et al. (2016) [35] propose 3DVO, introducing *Avoidance Planes*, where the obstacle is represented as the cross-section of the plane with the 3D obstacle. Instead of addressing all possible solutions in the velocity space, the algorithm evaluates planes P_ϕ , defined by the angle of rotation ϕ from the vehicle’s X-axis. Triangular or hyperbolic VO sections are considered more dangerous, while ellipsoidal sections are preferred when choosing the

avoidance plane. The next step is to select the avoidance velocity within that plane, following the VO principle of choosing a velocity outside the velocity obstacle cross-section. Tan et al (2020) [40] propose some improvements on the cross-sectioned obstacle equations and address static obstacles with Velocity Obstacle Pyramids, to address non spherical obstacles, such as tall buildings.

Choi et al. (2017) [36] address 3D Velocity Obstacle for low-cost micro UAVs. The authors propose methods to model the velocity obstacle using the limited sensing capabilities of these UAVs, estimating the obstacle’s radius, position, and velocity. The velocity command is chosen outside the union of velocity cones, following two heuristics: *To-Goal Search*, which selects a path aligned with the goal at maximum velocity or slower if needed, but still directed toward the goal; and *Maximum-Velocity Search*, which selects a path that prioritizes maximum speed even if it is not directly toward the goal. Liao et al. (2024) [41] transition to the elliptic domain for the agents’ shapes. The elliptic shape is able to model a more correct UAV’s shape, allowing a more accurate approximation of movement, and benefiting scenarios such as navigating near walls and decreasing detour distance.

3D extensions also allow the algorithm to be applied to Robotic Manipulators. Traditionally, this field of robotics uses Artificial Potential Fields. Vesentini et al. (2025) [42] models an arm as multiple agent, representing the base, elbow and end effector of an anthropomorphic robotic manipulator. To address the non spherical shape of the rod-shaped components of the manipulator, the authors propose creating velocity obstacles for the two ends, and defining the resulting VO as the convex set containing them.

The reviewed approaches demonstrate significant progress in extending Velocity Obstacles to 3D vehicles and environments. However, there are some issues with the existing approaches. The superposition of ORCA half-planes identified by Battisti and Muradore (2020) [34] can happen with closely spaced obstacles, as the authors identify, but is much more problematic when considering complex environments and one of the limitations of VO: modeling all obstacles as spheres. When considering obstacles that do not fit well inside a spherical model, the easiest approach would be to break down the obstacle into several spheres, but as discussed, closely spaced obstacles lead to deadlocks in ORCA.

Another limitation of the 3D extension of ORCA is that it is based on a maximum velocity that is the same in all directions. This does not accurately represent the behavior of multirotor UAVs, since their maximum velocities

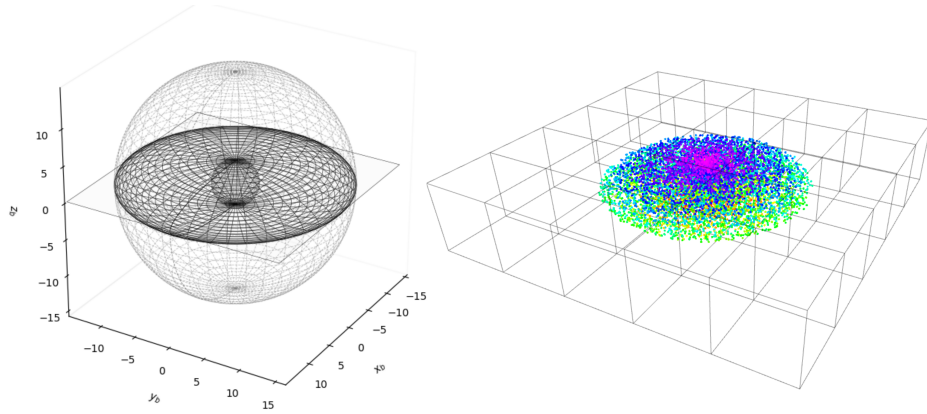


Figure 1: Illustration of two sphere models of an ellipsoidal velocity space in $[m/s]$, on the left, and an ellipsoidal velocity sample space, on the right, with the color representing the z velocity component, from red to purple

in the x and y directions are typically 3 to 5 times higher than in the vertical direction. As a result, the space of feasible velocities has the shape of an ellipsoid, instead of a sphere, as represented in Figure 1. To use a single maximum velocity parameter, either the sphere inscribed within the ellipsoid, or the sphere that encloses the ellipsoid must be considered, either not covering the full reachable velocity space, or inaccurately marking vertical velocities as feasible where they are not.

3DVO's avoidance planes decouple the problem into two: (i) choosing a plane and (ii) selecting an avoidance velocity within that plane. This reduction struggles with addressing closeness to obstacles. Although distance to the cross-sectioned obstacle (inside the plane) can be considered, distances to obstacles not intersected by the chosen plane cannot, leaving the closeness to these obstacles unaccounted for [35].

Choi et al. introduce a navigation cost to find 3D trajectories that follow a desired goal, but the search is restricted to two subspaces: the *To-Goal Search*, which reduces exploration to a line toward the goal, and the *Maximum-Velocity Search*, which reduces exploration to a spherical surface. By not evaluating the full sphere (directions and magnitudes simultaneously), the method exhibits oscillatory behavior [36].

This paper proposes a new approach to address 3D Velocity Obstacles, accurately representing feasible vehicle velocities and advancing VO towards safer robotic navigation in 3D dynamic environments.

To extend the VO to 3D and efficiently search for an avoidance maneuver within this space, the proposed method relies on sampling the vehicle’s control inputs into reachable linear velocity samples: linear velocities achievable within a small time interval, according to the vehicle’s kinematic limitations. This strategy avoids the complexity of defining continuous collision regions and kinematic constraints, which can lead to high-dimensional optimization problems. Unlike voxel- or grid-based approaches, which suffer from a trade-off between resolution and computational cost, velocity sampling provides a balanced alternative. It creates a finite and manageable set of candidate velocities that can be efficiently pruned using geometric and kinematic constraints, resulting in a discrete, collision-free set of feasible options.

In addition, this extension incorporates a heuristic based on the proximity to Velocity Obstacles and on the direction and magnitude of the desired velocity. The heuristic seeks a control input that is both safe and aligned with the navigation objectives defined by the higher-level navigation stack.

3. Sampling-based 3D Velocity Obstacles (S3VO)

This section describes the environment representation in a velocity space and the three steps that compose S3VO: Generating a set of feasible velocity samples, pruning them according to collisions, and evaluating and choosing an avoidance maneuver.

rephrase. provide introduction to the method (it's phases)

3.1. Velocity Space representation based on Obstacle Detection

The scene representation requires obstacles with static or dynamic nature to be represented in a velocity space, which must characterize the extrinsic features related to geometry and motion of all elements surrounding the vehicle that can be combined from heterogeneous perceptual systems. To develop a uniform representation principle, these elements are represented as spheres characterized by a radius, a position, and a velocity, along with the uncertainty associated with each parameter. The defined obstacle representation allows for the algorithm to perceive three-dimensional obstacles in dynamic environments.

One significant challenge in these environment representations is managing obstacles whose geometries are not easily approximated by a single sphere, such as elongated structures. In such instances, two primary strategies exist: modeling a larger bounding sphere that encompasses the obstacle, which results in a significant amount of conservatively occupied space, or

decomposing the obstacle into a sequence of smaller spheres. To maintain a representation that is geometrically faithful to the actual environment, the latter approach was adopted

3.2. Sampling-Based Obstacle Avoidance

The basic process for finding a safe velocity output consists of three steps: first, a finite set of control samples is generated from the kinematic constraints and mapped to linear velocity space; then, this set is pruned according to possible collisions following the Velocity Obstacle method; lastly, according to the desired velocity command and the obstacles, the remaining samples are evaluated using a multicriteria heuristic function.

Generating Samples

Each linear velocity sample is defined by: v_x, v_y, v_z . These linear velocities are obtained from the vehicle kinematics and represent the velocities that can be achieved within a given planning horizon Δt , considering limits on linear and angular velocities and accelerations, defining the set of Reachable Velocities (RV). Formally, let $\mathcal{U}$ denote the set of admissible actuator efforts $\mathbf{u}$, Δt the planning time interval, and $\mathbf{v}_A(t_0)$ the velocity of the agent A at the time t_0 . Then, with f representing a generic kinematic function that maps actuator efforts into resulting linear velocity, the set of reachable velocities is given by Equation 1, adapted from [5]:

$$RV(t_0 + \Delta t) = \{\mathbf{v} \mid \mathbf{v} = f(\mathbf{u}, \mathbf{v}_A(t_0), \Delta t), \mathbf{u} \in \mathcal{U}\} \quad (1)$$

The kinematics of heterogeneous vehicles modeled according to 12 values defines constraints on 6DoF. These values correspond to maximum linear accelerations, maximum angular accelerations, maximum linear velocities, and maximum angular velocities ($a_{x,\max}, a_{y,\max}, a_{z,\max}, \alpha_{\text{roll},\max}, \alpha_{\text{pitch},\max}, \alpha_{\text{yaw},\max}, v_{x,\max}, v_{y,\max}, v_{z,\max}, \omega_{\text{roll},\max}, \omega_{\text{pitch},\max}, \omega_{\text{yaw},\max}$). The minimum values are assumed to be the symmetrical counterparts of their maximum. These constraints are the basis for defining the set of admissible actuator efforts $\mathcal{U}$ and the function f . Sampling is performed in the set of admissible actuator efforts, following a uniform distribution in this space, and then mapped with the function f to create RV . In addition to the randomly generated samples, the vehicle's current velocity and the desired velocity are added, provided that both satisfy the kinematic constraints.

estas fases devem ser apresentadas logo no parágrafo que falta fazer ("rephrase") que indica as fases do método.

This sampling-based approach allows for structural versatility, as the sampling space and constraint-handling parameters are adaptable to heterogeneous platforms: By reconfiguring the 12-parameter kinematic set, the algorithm's search space transitions between 2D planar manifolds for surface vessels and 3D ellipsoidal volumes for aerial or underwater vehicles. Having linear velocities and accelerations in x , y , and z can model a 3D multicopter vehicle, searching in an ellipsoidal velocity space, and setting only x and y or x and yaw can deploy the algorithm for a surface vehicle, generating a velocity sample plane.

For the differential drive vehicle example, the mapping between velocities, from polar (v, ω_z) to linear velocities (v_x, v_y) , was performed according to the equation in 2. For holonomic vehicles, no mapping is needed.

$$v_x = v \cdot \cos(\omega_{yaw} \cdot \Delta t), \quad v_y = v \cdot \sin(\omega_{yaw} \cdot \Delta t), \quad v_z = 0. \quad (2)$$

Sample Pruning

The second step addresses the conditions introduced by the dynamic environment, pruning velocities that would result in a collision. This is done using the fundamental concept of the Velocity Obstacle approach. First, the spherical representation of each obstacle B is augmented with a Minkowski sum involving the spherical representation of the agent A . Each obstacle induces a Relative Collision Cone in A , denoted as CC_{AB} . This cone is defined as the set of velocities for agent A , reduced to a point $\hat{A}$, that would result in a trajectory intersecting B . Geometrically, this corresponds to an infinite-height cone with its vertex at $\hat{A}$, tangent to the augmented obstacle $\hat{B}$. Finally, the Velocity Obstacle induced by B on agent A corresponds to translating the Relative Collision Cone by the obstacle's velocity, v_B . Similar to other implementations [30], a time horizon simplification is made on velocities that, although leading to a collision, do so after a defined threshold Time Horizon. This is done by calculating the minimum time to collision with a given obstacle B . Specifically, the distance d_B between the agent and the center of obstacle B , the combined radius $r_B + r_A$ accounting for both the obstacle's and the agent's size, and the relative velocity v_{AB} between the two entities. The time to collision is estimated and compared with a time threshold to decide whether the sample is considered safe.

Figure 2 depicts the obstacle representation and its Minkowski sum with the agent's size, the Relative Collision Cone, the Velocity Obstacle, and the agent and obstacle velocity, as well as the Time Horizon truncation.

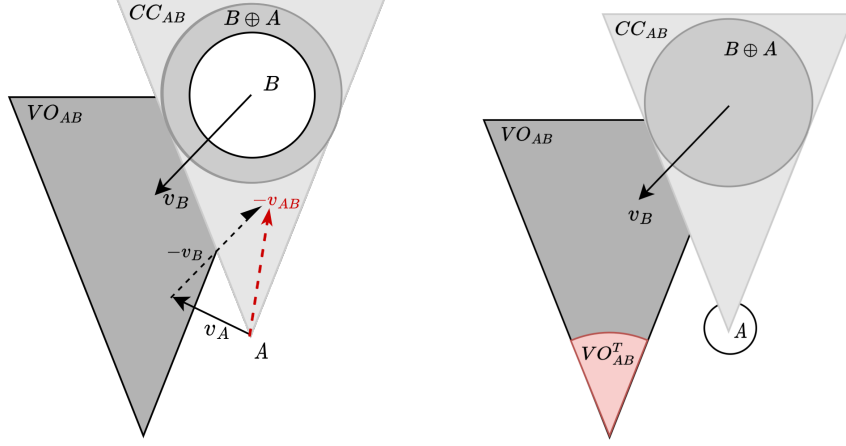


Figure 2: Comparison between full and truncated Velocity Obstacle representations. Left: Moving Obstacle, Relative Collision Cone, and Velocity Obstacle. Right: Truncated Collision Cone.

By definition, the velocities within VO_{AB} are the set of velocities that lead to a collision. As such, the set of samples generated in the previous step is pruned, creating the set of feasible non-colliding velocities. In practice, this pruning is done by verifying a set of geometric conditions.

The translated velocity is denoted as the relative velocity $\vec{v}_{AB}$. The effective obstacle radius of obstacle B is the sum of its radius r_B , the radius of the vehicle r_A , and an additional protective zone $r_{\text{protective}}$. The distance between the agent and obstacle B is denoted as d_{AB} .

Equation 4 represents the condition under which a potential collision is detected. Let p be the projection of $\vec{v}_{AB}$ onto the cone axis. The left-hand side term corresponds to the perpendicular distance from $\vec{v}_{AB}$ to the cone axis, obtained using the Pythagorean theorem, while the right-hand side term represents the radius of the cone at height p , since the radius at the obstacle is given by r_B . Thus, when the perpendicular distance is smaller than this radius, the relative velocity lies inside the cone, indicating a possible collision.

The time to collision is estimated according to Equation 5 and compared with a time to collision threshold to decide whether the sample is considered safe.

$$p = \frac{\vec{v}_{AB} \cdot \vec{x}_B}{\|\vec{x}_B\|}, \quad (3)$$

$$\sqrt{\|\vec{v}_{AB}\|^2 - p^2} < p \cdot \frac{r_B}{d_{AB}}, \quad (4)$$

$$t_{\text{collision},B} = \frac{d_{AB} - (r_B + r_A)}{v_{AB}}, \quad (5)$$

Formally, considering the planning time horizon $t_{\text{collision,max}}$, the Velocity Obstacle VO_{AB} is defined as the set of agent velocities satisfying both the collision condition and the time constraint following Equation 6. The left and right side of equation 4 are denoted as r_{actual} and r_{actual} , respectively .

$$VO_{AB} = \{\mathbf{v} \mid r_{\text{actual}}(\mathbf{v}) < r_{\text{expected}}(\mathbf{v}) \wedge t_{\text{collision},B}(\mathbf{v}) < t_{\text{collision,max}}\}, \quad (6)$$

Considering the set of detected obstacles $\mathcal{B} = \{B_1, B_2, \dots, B_N\}$, the combined Velocity Obstacle VO_A is defined as the union of the individual velocity obstacles generated by each obstacle. Then, the set of Reachable Avoidance Velocities within Δt (RAV), defined as the subset of reachable velocities not leading to collision with any detected obstacle within the planning horizon, is given by equation 7:

$$RAV(t_0 + \Delta t) = RV(t_0 + \Delta t) \setminus \bigcup_{B_i \in \mathcal{B}} VO_{AB_i}. \quad (7)$$

Sample Cost Heuristic

Each sample is evaluated using a multicriteria heuristic cost function [30]. The first component is a danger cost, which penalizes maneuvers close to a Velocity Obstacle, reflecting the risk of leading to a collision. The second is a navigation cost that penalizes heading, velocities that deviate from the desired direction, and magnitude, velocities that do not match the desired velocity magnitude. Lastly, a momentum cost, that penalizes velocity variation is added.

The output velocity is the sample with the lowest total cost according to the defined weights. Figure 3 shows the heading cost (left), the magnitude cost (center) and the danger cost for a static obstacle (right).

The direction error, C_{dir} , quantifies how well the sample velocity vector $\mathbf{v}_s = (v_{sx}, v_{sy}, v_{sz})$ aligns with the desired velocity vector $\mathbf{v}_d = (v_{dx}, v_{dy}, v_{dz})$.

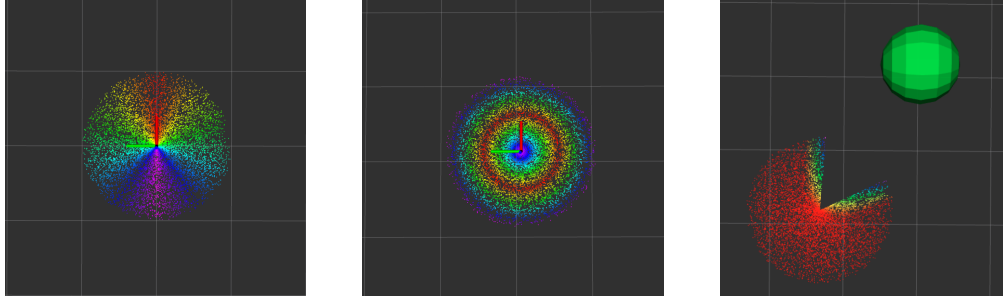


Figure 3: Examples of cost functions in the velocity space $[m/s]$ with a grid size of $1 [m/s]$: left — heading cost based on angle difference; center — magnitude cost based on velocity error; right — danger cost around a moving obstacle. Cost values go from low (red) to high (purple).

It is calculated using the dot product of their normalized vectors, $\hat{\mathbf{v}}_s \cdot \hat{\mathbf{v}}_d$. The direction error is then normalized to the range $[0, 1]$ by inverting and scaling the dot product, as shown in equation 8:

$$C_{dir} = \frac{1 - (\hat{\mathbf{v}}_s \cdot \hat{\mathbf{v}}_d)}{2} \quad (8)$$

The magnitude error (C_{mag}) quantifies the absolute value of the difference between the sample and desired velocity magnitude. This error is calculated from the absolute value of the differences of the norms of the velocity vectors. To ensure consistent scaling across all operating conditions, this error is normalized by the vehicle’s maximum velocity (v_{max}) as shown in the following equation 9:

$$C_{mag} = \frac{\Delta_{mag}}{v_{max}} = \frac{||\mathbf{v}_s|| - ||\mathbf{v}_d||}{v_{max}} \quad (9)$$

The *danger* component of the cost function is a risk-aware penalty applied to velocity samples that are close to but not strictly inside a Velocity Obstacle (VO). This cost helps to avoid tangential movements near obstacles, whether static or dynamic. It supports a preemptive collision avoidance behavior under uncertainty in velocity and position estimates, as proposed by [30].

This penalty is applied when the velocity sample lies near the boundary of the collision cone. In particular, if the projection of the relative velocity onto the obstacle direction is positive, indicating motion toward the obstacle, and the difference between actual and expected radial offsets is smaller than a threshold r_{thresh} , a cost is added to the sample. This cost is computed according to Equation 10:

$$C_{\text{danger}} = \left(r_{\text{max}} - \left(\sqrt{\|\vec{v}_{AB}\|^2 - p^2} - p \cdot \frac{r_B}{\|\vec{x}_B\|} \right) \right) \cdot \frac{p}{r_{\text{max}}}. \quad (10)$$

Equation 10 distributes the danger cost, penalizing two factors: velocities that are closer to the velocity obstacle, and velocities that more quickly approach an obstacle. Firstly, following [30], velocities that lie closer to the cone boundary are penalized more strongly than those at the outer threshold, within a range defined by r_{max} , and scaling linearly from no cost at the boundary of the range to maximum cost at the velocity cone boundary. This component reflects the cost of navigating close to a velocity obstacle. Secondly, the forward projection p scales the cost with distance along the cone axis, so that velocities with larger forward components toward the obstacle are penalized more heavily, reflecting the risk of a faster approach.

Lastly, to minimize oscillation, a momentum cost C_{mom} is added. It penalizes the angular deviation between the current velocity $\hat{\mathbf{v}}_{\text{curr}}$ and the sample velocity $\hat{\mathbf{v}}_s$:

$$C_{\text{mom}} = \frac{1 - (\hat{\mathbf{v}}_s \cdot \hat{\mathbf{v}}_{\text{curr}})}{2} \quad (11)$$

The final velocity is chosen by minimizing the total weighted cost $C\mathbf{v}_s$:

$$C(\mathbf{v}_s) = (1 - w_{\text{danger}}) \cdot (w_{\text{dir}}C_{\text{dir}} + w_{\text{mag}}C_{\text{mag}} + w_{\text{mom}}C_{\text{mom}}) + w_{\text{danger}}C_{\text{danger}} \quad (12)$$

4. Results

In this results section, S3VO is first used to model a simulated multirotor UAV. The 3D implementation is compared with RVO2-3D³ (ORCA 3D).

Secondly, this section presents the results applying S3VO to surface vehicles. In a simulated environment, two types of 2D vehicles: holonomic and diff-drive are presented, and lastly the algorithm is deployed on a real surface vehicle and qualitatively validated in a real-world setup at Barragem de Alqueva, Portugal.

In all experiments, the navigation stack was implemented in ROS2 according to the following pipeline: first, a fixed goal pose was defined (typically

³<https://gamma.cs.unc.edu/RVO2/downloads/>, accessed on October 15, 2025

at $(10, 0, 0)$); second, a global planner generated a velocity vector directed toward the goal, utilizing a constant magnitude velocity followed by a linear deceleration upon approach; third, the obstacle avoidance algorithm; and finally, a velocity filter was applied between the avoidance output and the Gazebo plugins to enforce kinodynamic constraints on velocity and acceleration specific to each setup.

4.1. 3D Multirotor Simulation

This section presents the preliminary results obtained in a set of 4 simplified Gazebo simulated scenarios, comparing S3VO with ORCA. frase a identificar os cenários e sua pertinência para a avaliação

In scenarios S1 the goal is located behind a single static obstacle. In scenario S3, the agent has to go around a moving platform, modeled as a plane of 6 spherical obstacles. In scenarios S4 and S5, the agent starts near the goal position, with 4 dynamic obstacles converging to the goal, forcing the agent to step away and come back: In S4, these 4 obstacles are modeled as small spheres, and S5 presents taller obstacles modeled as a pair of stacked spheres. A video presenting the results of this algorithm comparison can be accessed using the link in the footnote⁴.

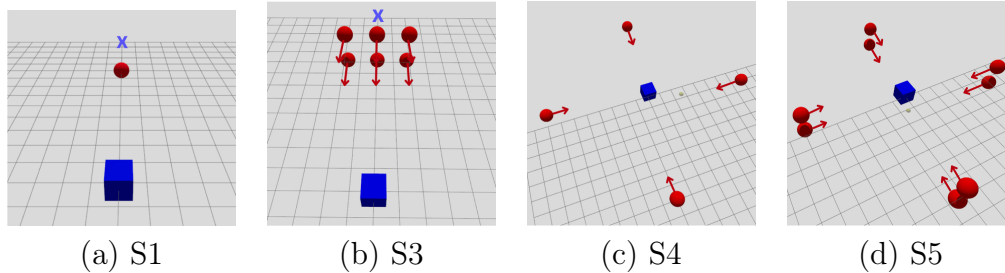


Figure 4: ORCA and S3VO 3D Gazebo Scenarios. nada é simples num artigo. Está a inferiorizar-te

These simplified scenarios present the main differences observed in the behaviour of the two algorithms: The deadlocks found for ORCA in scenarios S1 and S3, as can be observed in the trajectory plots of figures 5 and 6, are similar to the deadlock behavior identified by Battisti and Muradore (2020) [34]. In Scenario S1, equivalent to the deadlock "Agent Alignment", the desired velocity of the agent is aligned with the obstacle, resulting in a colinear velocity correction that gradually slows down the agent, never

⁴https://youtu.be/_OoPVpV6XLg

[passar verde para cima. ver comentário dos cenários](#)

choosing a direction to cross it. Scenario S3, corresponding to the deadlock "Motion through close agents", occurs because ORCA considers individual half-planes for each obstacle, and their superposition causes the chosen velocity to be backwards, never overtaking the obstacles. For S3VO, the intrinsic randomness of the particles and the heuristic cost function lead to a direction being chosen in order to progress towards the goal in S1. In Scenario S3, instead of velocity vectors created by obstacle half-planes canceling out, the cost of navigating close to each obstacle is directly considered and summed, and the heuristic function chooses a velocity outside the velocity obstacles that allow for progress in the direction to the goal.

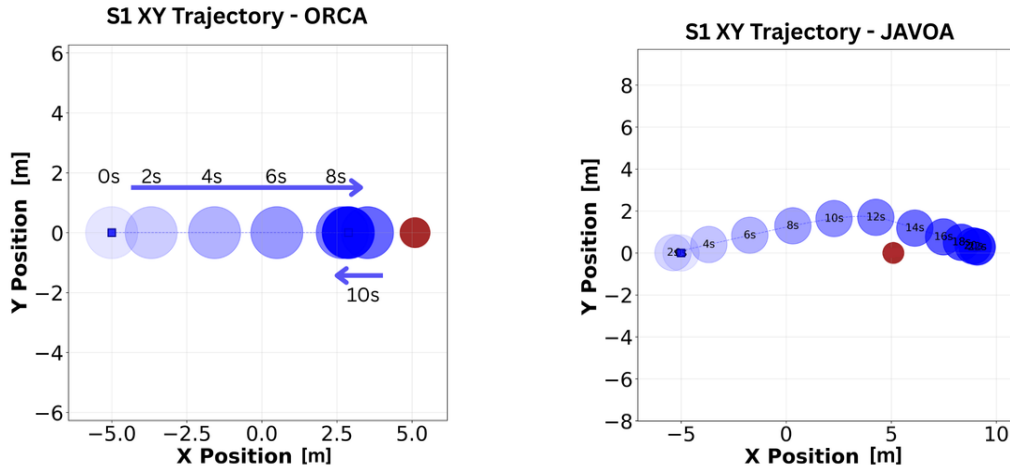


Figure 5: Multirotor Vehicle Scenario S1 - XY trajectory in meters $[m]$ over time, with markers indicating timestamps in seconds $[s]$: Left - ORCA; Right - S3VO

The resulting trajectories for Scenario S4, in Figure 7 present successful obstacle avoidance for both algorithms, but highlight different behaviours. For ORCA, the horizontal effects cancel out, due to the symmetric nature of the scenario: The avoidance maneuver is purely vertical, at maximum downward velocity, as can be seen in the velocity profiles of Figure 8, ORCA commands a z-velocity component higher than the vehicle limitations, that in a real UAV might lead to a collision. S3VO finds a diagonal maneuver with velocity components in x , y , and z , meeting the velocity limits for all of them.

Scenario S5 presents a similar scenario, but now the two half-spaces created by each obstacle lead ORCA to a deadlock in the horizontal plane, and

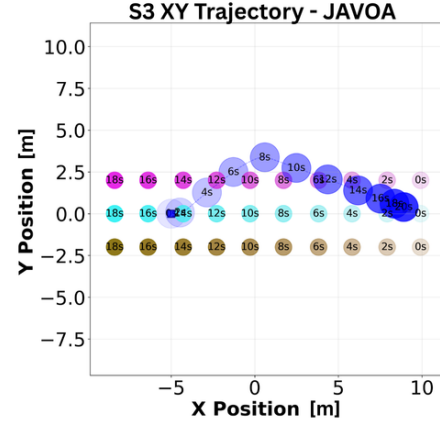
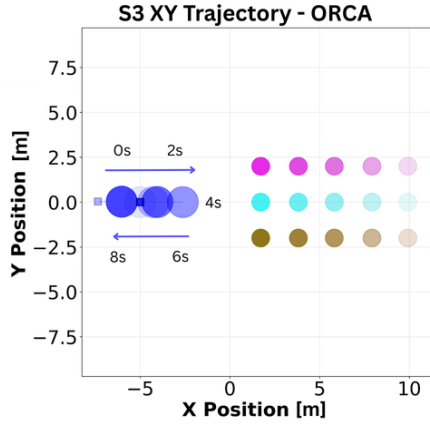


Figure 6: Multirotor Vehicle Scenario S3 - XY trajectory in meters $[m]$ over time, with markers indicating timestamps in seconds $[s]$: Left – ORCA; Right – S3VO

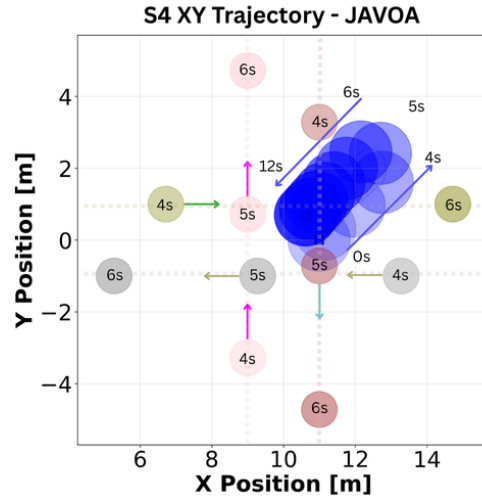
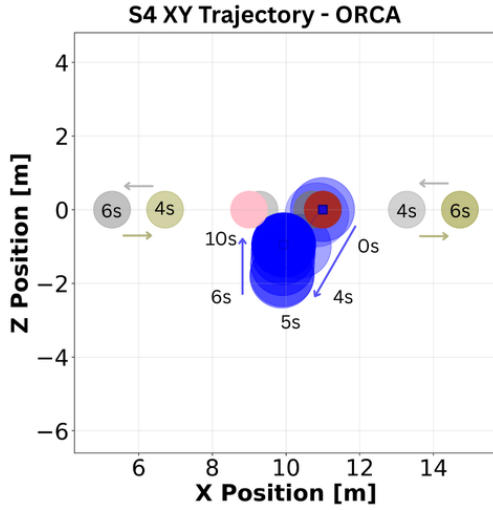


Figure 7: Multirotor Vehicle Scenario S4 - Trajectory in meters $[m]$ over time, with markers indicating timestamps in seconds $[s]$: Left – ORCA (XZ Trajectory); Right – S3VO (XY trajectory)

tenta discussão Quantitativa sempre que possível! Não deixe À itnerpretação de cada um --> faz highlight sempre do bom do método que está a ser proposto.

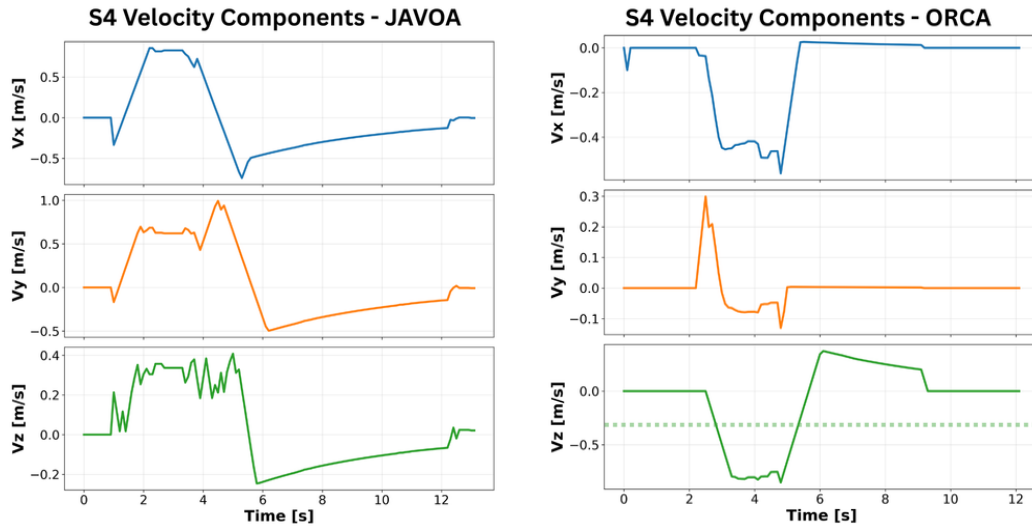


Figure 8: Multirotor Vehicle Scenario S4 - Velocity Components in meters per second $[m/s]$ over time, dotted arrow marking downward velocity limit: Left – ORCA; Right – S3VO

the taller obstacles don't allow for a feasible vertical avoidance maneuver, resulting in a collision for ORCA, as can be seen in Figure 9. S3VO, similarly to the previous scenario, finds a diagonal maneuver that avoids the obstacles in the horizontal plane, avoiding the collision.

4.2. 3D Randomized Trials

sei que foi indicação do revisor, mas o titulo "randomized" é tão horrível.

In order to evaluate the performance of the proposed algorithm, simulation tests were performed for the UAV using randomized scenarios. rephrase

In the implemented setup, the agent is positioned at $(-10, 0, 0)$ with a goal at $(10, 0, 0)$. Between the agent and the goal, two types of obstacles are spawned:

- **Static Obstacles:** modeled as clusters of 4, 6, or 8 spherical obstacles.
- **Dynamic Obstacles:** modeled as 1 or 2 spherical obstacles.

The scenarios contain a minimum of 10 and a maximum of 50 obstacles between the starting and goal positions. Each spherical obstacle has a radius of 1 m. Both static and dynamic obstacles are spawned randomly between $x = -5$ and $x = 8$, ensuring no obstacles spawn on top of the agent or

muito difícil de visualizar os "ganhos". Procura adicionar o um screen time da experiência em 3D para facilitar a interpretação. Os gráficos com a legenda devem ser autoexplicativos sempre.

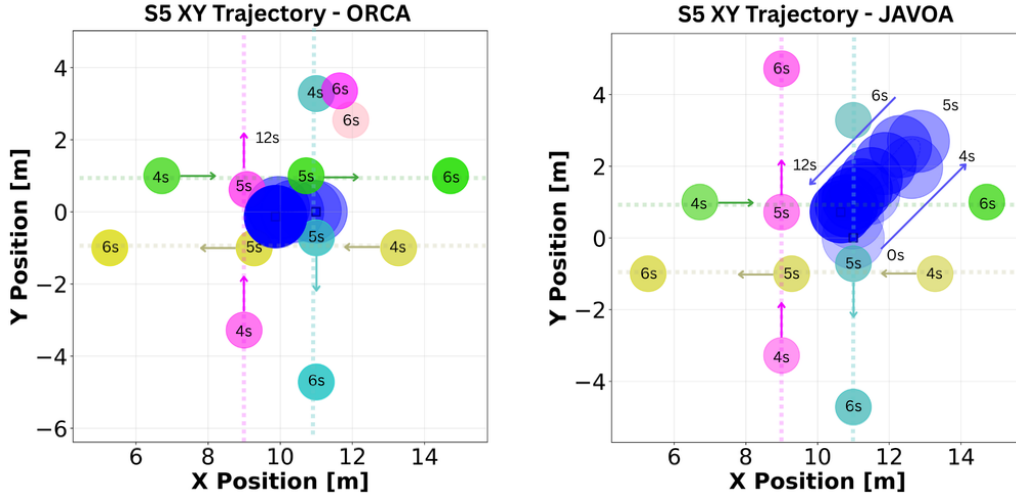


Figure 9: Multirotor Vehicle Scenario S5 - XY trajectory in meters [m] over time, with markers indicating timestamps in seconds [s]: Left – ORCA; Right – S3VO

the goal. Dynamic obstacles are assigned a random velocity between 0 and 4 m/s .

The decision to **utilize** obstacle clusters instead of fully random scenarios is based on environmental realism and the limitations of this simplified geometric model. In real-world environments, the approximation of static obstacles as spheres is rarely the most accurate representation. Structural elements such as walls, buildings, lamp posts, or containers typically possess elongated or flat geometries that are more effectively modeled as clusters of obstacles arranged in lines or planes. Using a single, large bounding sphere to encompass these structures represents an overly conservative approach. This significantly reduces the navigable space, often leading to the generation of inflated Velocity Obstacles (VO) that may cause a planner to incorrectly identify a path as blocked. In maritime scenarios, even dynamic obstacles, such as ships, can not be modeled by single conservative spheres, as shown in the examples illustrated in Figure 10.

Furthermore, evaluating an algorithm's performance in a fully random environment fails to capture these critical structural patterns. In a fully random setup, the probability of spawning adjacent elements with identical velocity vectors and positions is nearly zero. By explicitly designing clusters, the algorithm's ability to navigate these environments is tested.

This setup models a multirotor UAV, considering its velocities along dif-

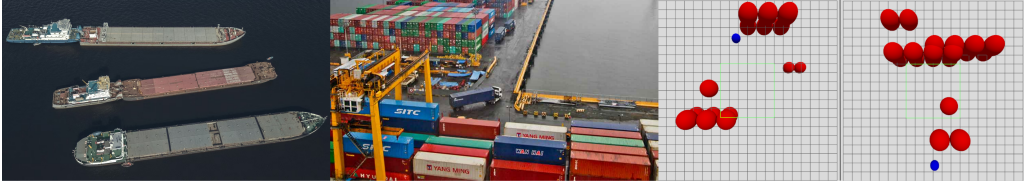


Figure 10: Real-World Obstacle Scenarios (left) vs two randomly generated obstacle scenario examples (right). Photos by Siarhei Palishchuk and Jerome Monta via Unsplash.

ferent axes. The maximum velocities were based on the M1200 T-Drone UAV specifications. The vehicle has a maximum ascent speed of 5 m/s, a maximum descent speed of 3 m/s, and a maximum horizontal flight speed of 15 m/s according to the vehicle documentation⁵. Additionally, maximum accelerations of 1 m/s² were considered in all directions. For both ORCA and S3VO, the agent radius was set as 0.5 m, time horizon threshold of 5 seconds. The S3VO parameters set a planning horizon Δt of 3.0 s, a radius threshold of 0.5 m for the danger cost margin, and a sample size of 10,000. For the heuristics function, the danger_weight was set to 0.80, and the navigation cost for heading and magnitude were set as 0.9 and 0.1, with a momentum cost of 0.1. The UAV maximum velocities in x , y and z are set to 15, 15, 5 m/s, respectively. The ORCA parameters model the UAV as a sphere with max velocity 15 m/s. The described parameters are summarized in Table 1

The results highlight the identified limitations of ORCA when navigating closely-spaced obstacles. When the agent encounters a cluster of obstacles between its current position and the goal, the velocity vector calculated by ORCA fails to commit to a specific direction. Consequently, the agent approaches the obstacle at a reduced considering the time horizon constraint, without crossing it.

In the experimental setup featuring wall-like obstacles, this behavior led to a failure to reach the goal in 56 out of 100 scenarios, as illustrated in Figure 11. In the remaining 44 cases, success was only achieved when the offset between the obstacles and the direct line-to-goal was sufficient for the algorithm to choose a side, or when interactions with dynamic obstacles repositioned the agent onto a clear path.

In contrast, S3VO successfully reached the goal without collisions in 100

⁵<https://shop.tmotor.com/>, accessed on October 14, 2025

Table 1: Comparison of Navigation Parameters: S3VO vs. ORCA

Parameter	S3VO	ORCA
Obstacle Avoidance Layer		
Neighbor Distance	50 m	50 m
Max Neighbors	50	50
Time Horizon Threshold	5.0 s	5.0 s
Agent Radius	0.5 m	0.5 m
Max Speed (v_x, v_y, v_z)	15, 15, 5 m/s	15, 15, 15 m/s
Loop Frequency	10 Hz	10 Hz
Planning Horizon (Δt)	3.0 s	-
Radius Threshold	0.5 m	-
Number of Samples	10,000	-
Heuristics weights		
Danger Weight	0.80	-
Heading Cost	0.90	-
Magnitude Cost	0.10	-
Momentum Cost	0.10	-
Global Planner Layer		
Desired Velocity (V_{des})	5.0 m/s	5.0 m/s
Goal Approach	Linear deceleration when distance < 10m	

out of 100 scenarios. While its behavior remains similar to ORCA when navigating individual obstacles—showing no specific preference for horizontal or vertical crossing—S3VO consistently calculated a vertical velocity component to overcome static obstacles blocking the path to the goal.

The plots and results in this section are evaluated with the following metrics, where the (n) exponent refers to the time step:

- **Obstacle Clearance:** Minimum distance between the center of the agent and the center of each obstacle, subtracting the combined radii of the agent and the obstacle (1.5 m)

passar para a metodologia

$$C_{min} = \min_{n,i} (d_{AB_i}^{(n)} - r_A - r_{B_i}) \quad (13)$$

- **Distance Traveled:** The total distance D is computed only for scenarios where the agent reaches the goal successfully. It is defined as the cumulative sum of Euclidean distances between consecutive agent

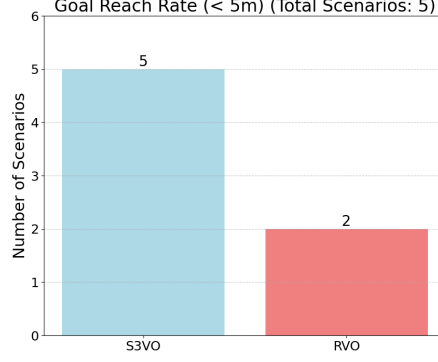


Figure 11: Ente

positions:

$$D = \sum_{n=1}^N \sqrt{(x_A^{(n)} - x_A^{(n-1)})^2 + (y_A^{(n)} - y_A^{(n-1)})^2 + (z_A^{(n)} - z_A^{(n-1)})^2} \quad (14)$$

- **Time to Reach Goal:** The total time T required for the agent to reach the target location. This is recorded as the final timestamp $t^{(N)}$ of the simulation:

$$T = t^{(N)} - t^{(1)} \quad (15)$$

Similar to the distance metric, this is only recorded for successful scenarios where the goal condition is met.

- **Path Smoothness:** This metric (m_{vsm}) evaluates trajectory quality by measuring the mean magnitude of acceleration. A lower value indicates a smoother path with fewer abrupt velocity changes. It is calculated using the following equation [43]:

$$m_{\text{vsm}} = \frac{1}{N-1} \sum_{n=1}^{N-1} \frac{\|\mathbf{v}_A^{(n+1)} - \mathbf{v}_A^{(n)}\|}{t^{(n+1)} - t^{(n)}} \quad (16)$$

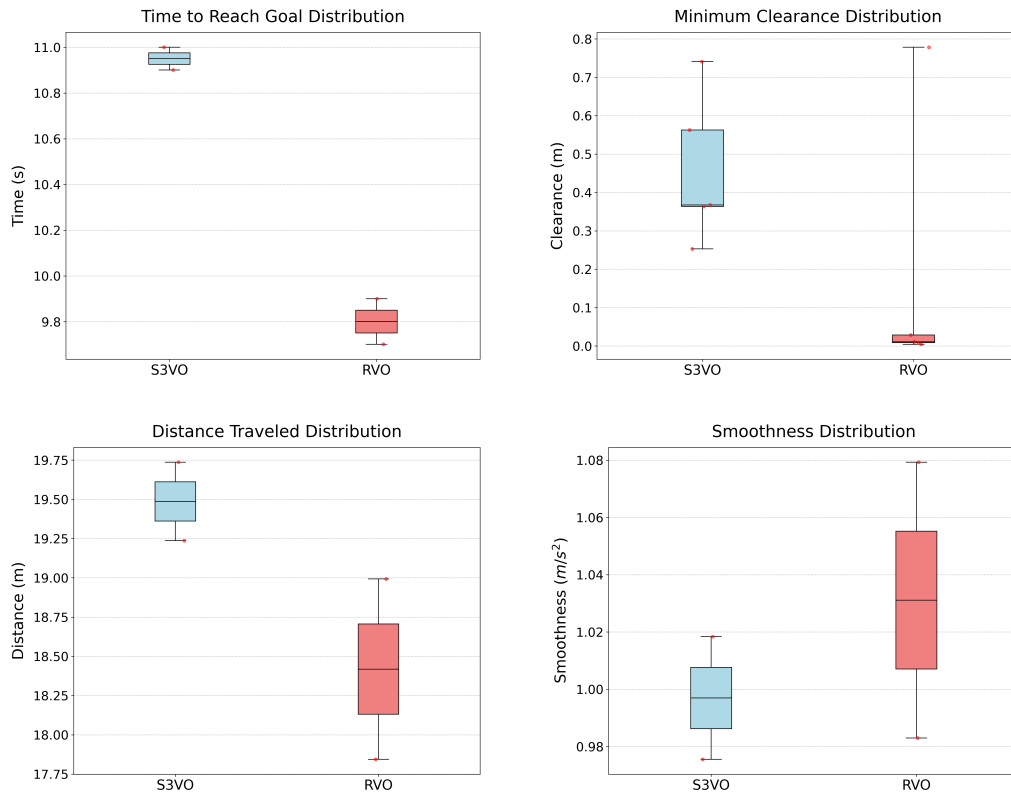


Figure 12: Comparative performance results for S3VO and RVO: Time to Reach Goal (top-left), Minimum Obstacle Clearance (top-right), Total Distance Traveled (bottom-left), and Path Smoothness (bottom-right).to be redone after new sim, and for improved visual

Randomized Trials Examples

Placeholder graphs. Sims to be re-run and plots to be enhanced The results depicted in Figure 13 highlight a case of deadlock for the ORCA algorithm. Under the same conditions, S3VO avoided the obstacles by navigating above them. The corresponding clearance plots for each obstacle are also provided.

Figure 14 illustrates an additional scenario where both algorithms achieved the goal. Although both opted to pass over the obstacles, S3VO exhibited superior performance by maintaining a larger clearance margin compared to the baseline

S3VO Ablation

To compare the impact of the proposed safety velocity margin, the same 100 scenarios were tested for an ablated S3VO version. For this purpose, the `radius_margin` and the `danger_cost` parameters were set to zero. The remaining parameters were set with the same values as the ones for S3VO in Table 1.

As shown in Figure 15, the absence of the danger cost leads to a noticeable increase in a collision rate. The proposed danger cost enhances safety, providing a safety margin not around the obstacle in the traditional sense, but around the velocities that might lead to a collision.

4.3. 2D Vehicle Simulation

This section demonstrates the capability of this 3D-environment algorithm to remain applicable to 2D vehicles by adjusting the vehicle’s maximum velocity and acceleration parameters. For this purpose, a simple scenario is presented for the navigation of a 2D holonomic vehicle and a differential drive vehicle. Under the same setup, the Artificial Potential Fields method was implemented for comparison, with the following parameters: attraction strength of 100, repulsion strength of 10, and maximum influence distance of 3 m.

The testing environment consists of a simple simulation setup in the *Gazebo* simulation environment, with 1 meter diameter spherical obstacles. The overall pipeline used takes as input a velocity vector pointing towards the goal, set to $[10, 0, 0]$ m, with a fixed magnitude of 1 m/s.

The S3VO parameters set all vehicles with a radius of 1.4 m, a planning horizon Δt of 3.0 s, a radius threshold of 0.1 m for the danger cost margin, a maximum time to collision of 120 s, a protective zone of 0.5 m, and a sample

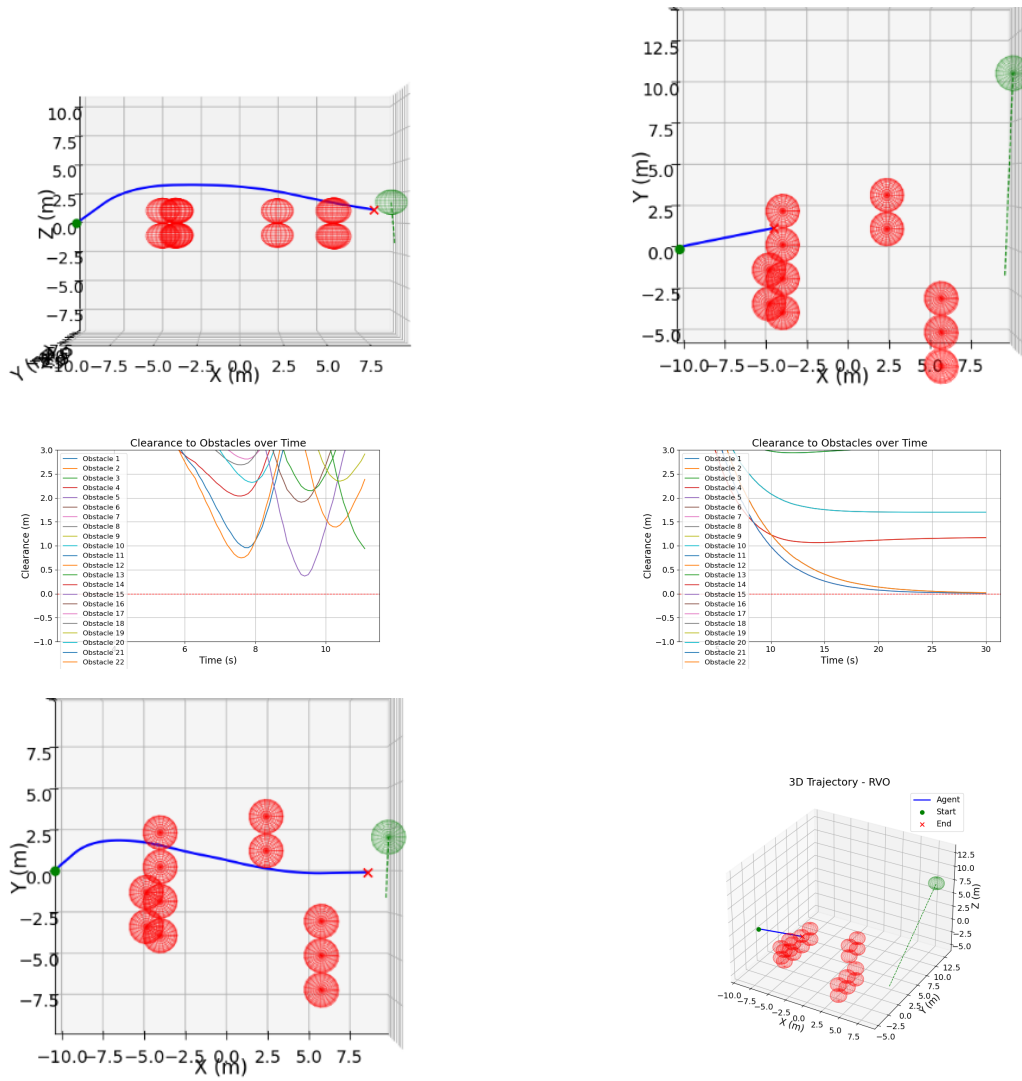


Figure 13: Comparative analysis of Scenario 000: Side views (top row), clearance over time plots (middle row), and S3VO top/isometric views (bottom row). S3VO results are shown on the left and ORCA results on the right where applicable. Placeholder graphs. Sims to be re-run and plots to be enhanced

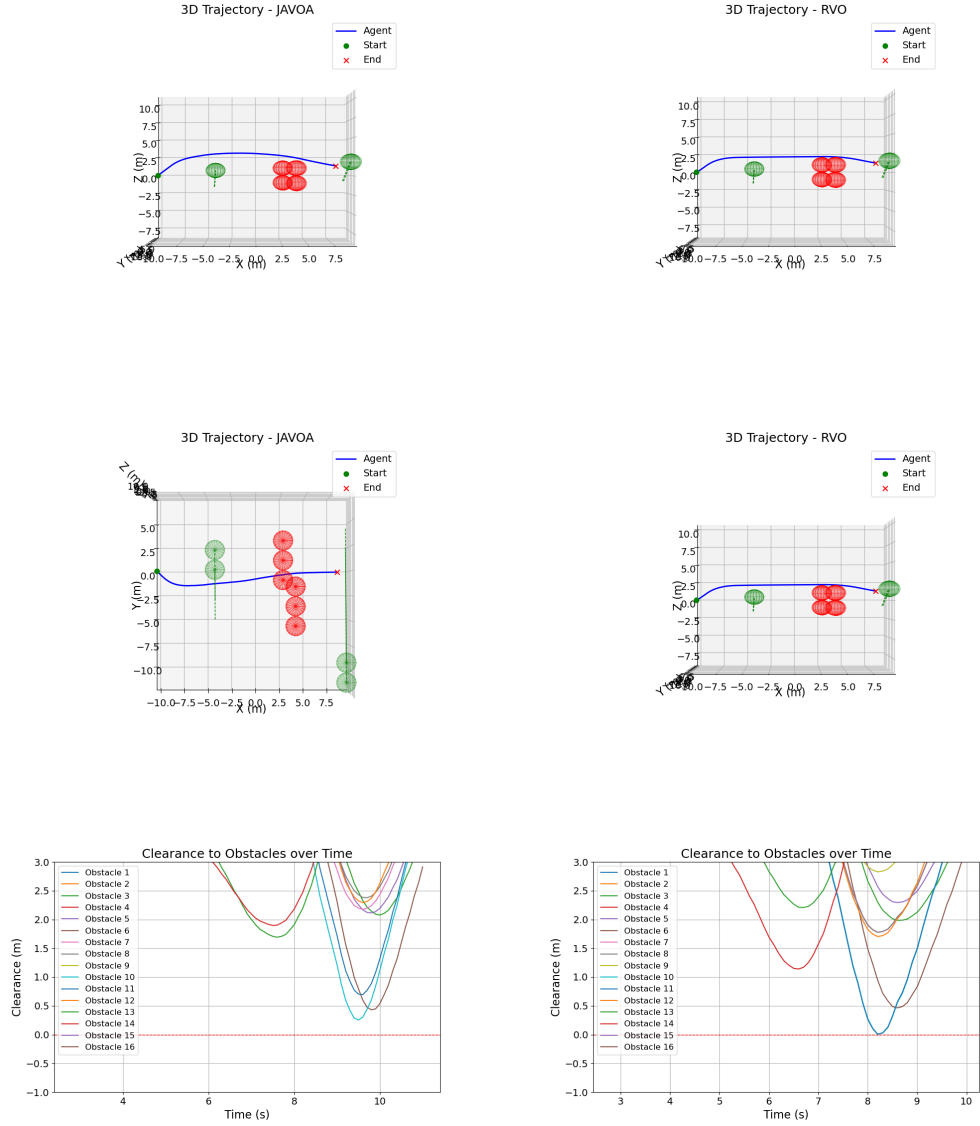


Figure 14: Comparative analysis of Scenario 001: Side views (top row), top views (middle row), and clearance over time (bottom row). S3VO results are shown on the left and RVO results on the right. Placeholder graphs. Sims to be re-run and plots to be enhanced

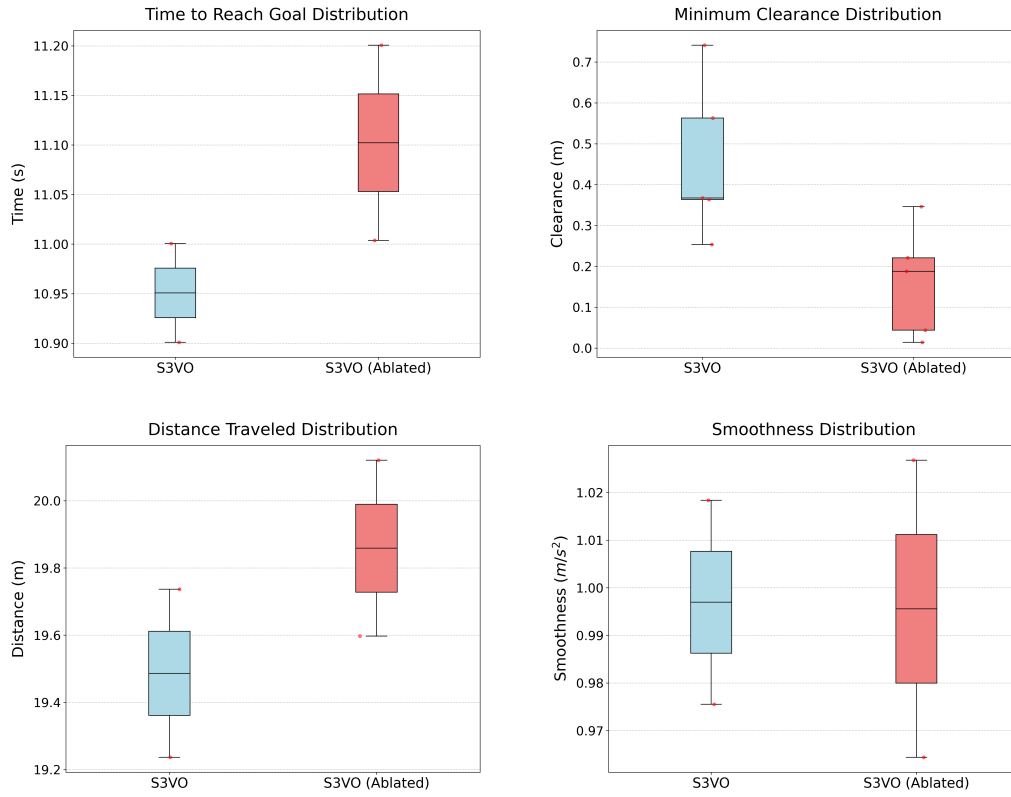


Figure 15: Ablation study performance results: Time to Reach Goal (top-left), Minimum Obstacle Clearance (top-right), Total Distance Traveled (bottom-left), and Path Smoothness (bottom-right) to be redone after new sim, and for improved visual comparing with normal one.

size of 10,000. For the heuristics function, the danger_weight was set to 0.90, and the navigation cost for heading and magnitude were set as 0.7 and 0.3 for the differential drive vehicle and 0.5 and 0.5 for the 2D holonomic.

Table 2: Simulation Parameters for Holonomic and Diff-Drive Vehicles. Linear velocities v in [m/s], angular velocities ω in [rad/s], linear accelerations a in [m/s²], and angular accelerations α in [rad/s²].

Vehicle	v_x^{max}	v_y^{max}	v_z^{max}	ω_{roll}^{max}	ω_{pitch}^{max}	ω_{yaw}^{max}
Holonomic	1.0	1.0	0.0	0.0	0.0	0.0
Diff-Drive	1.0	0.0	0.0	0.0	0.0	1.0

Vehicle	a_x^{max}	a_y^{max}	a_z^{max}	α_{roll}^{max}	α_{pitch}^{max}	α_{yaw}^{max}
Holonomic	1.0	1.0	0.0	0.0	0.0	0.0
Diff-Drive	1.0	0.0	0.0	0.0	0.0	1.0

A scenario with a single dynamic obstacle is presented: the obstacle moves perpendicularly to the agent's trajectory, intersecting it at $x = 5$ m, at $t = 13$ s. The resulting trajectories of the robot and the obstacle over time and velocity component profiles are shown in Figure 16 for the holonomic 2D vehicle, and in Figure 17 for the Diff-Drive 2D vehicle.

Table 3: Metrics Comparison between S3VO and APF for Holonomic and Diff-Drive Vehicles

Metric	S3VO	APF
Holonomic Vehicle		
Path Length	15.21 m	18.58 m
Path Elongation	1.06×	1.26×
Min. Clearance	0.69 m	1.00 m
Diff-Drive Vehicle		
Path Length	15.58 m	24.08 m
Path Elongation	1.06×	1.61×
Min. Clearance	0.99 m	0.95 m

In order to

The second validation phase used a more complex setup for a differential-drive USV, NEST [37][38]. This vehicle has a catamaran-style hull and a vectored configuration that enables differential-drive behaviour. First, a Gazebo

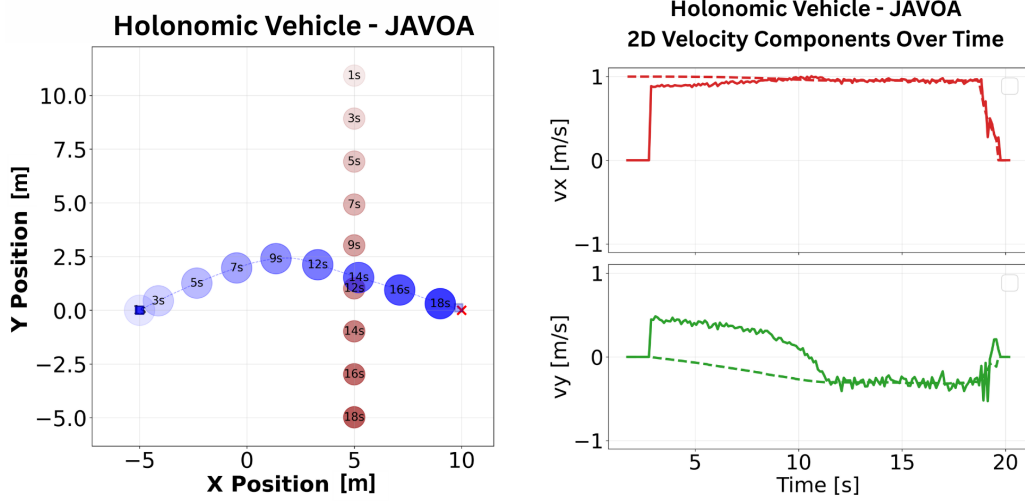


Figure 16: Holonomic Vehicle (S3VO): Left – XY trajectory in meters [m] over time, with markers indicating timestamps in seconds [s]; Right – velocity components in [m/s] over time [s]. Robot trajectory in blue, obstacle trajectory in red.

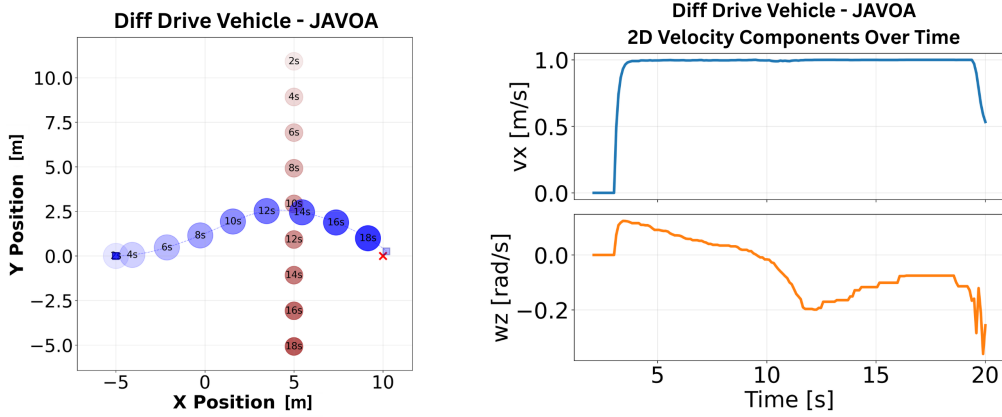


Figure 17: Diff-Drive Vehicle (S3VO): Left – XY trajectory in meters [m] over time, with markers indicating timestamps in seconds [s]; Right – linear and angular velocity components in [m/s] and [rad/s] over time [s]. Robot trajectory in blue; obstacle trajectory in red.

simulation environment with water drag and buoyancy dynamics, increasing

complexity and realism⁶, and then real-world testing in Barragem de Alqueva, Portugal.

The NEST vehicle followed the same kinematic principles as the differential-drive case but with specific adjustments: a larger radius of 2.15 m, a shorter planning horizon of 1.0 s, a larger danger cost threshold of 0.2 m, a reduced sample size of 3,000, and a lower danger_weight of 0.70.

The vehicle was controlled through a force topic, for which a PID control layer was integrated. This topic contains the desired force and moment in 3 degrees of freedom: F_x , F_y , and M_z , which are then mapped to the vehicle configuration.

In the simulation stage, two main scenarios (labeled as D0 and D1) are presented and set an obstacle moving perpendicularly to the agent’s velocity, intersecting it at $x = 5$. The difference between these scenarios lies in the initial position of the obstacle, causing it to intersect the trajectory at $t = 9$ s in D0 and at $t = 11$ s in D1. The goal of this scenarios was to evaluate the feasibility of the algorithm to model real-world vehicle interactions of crossing and giving way to another surface vehicle, as represented in the trajectories of Figure 18.

This more realistic vehicle control highlights some of the problems that occur with real vehicles, as they are not capable of reaching exactly and instantaneously a desired velocity. This leads to velocity components plots, like those presented in Figures 19, where, especially in the v_x velocity, there is a discrepancy between the commanded velocity and the velocity actually achieved by the vehicle. The safety velocity margin around the velocity obstacles added by the Danger Cost is precisely designed to handle this type of problem. A velocity that is not inside a velocity obstacle but is very close to it is penalized, representing the risk that, if that velocity is commanded, the control followed might introduce some deviations and lead to a collision.

Finally, the implemented approach was tested in a real aquatic environment using the actual NEST surface vehicle in Barragem de Alqueva, Portugal. This final test was intended as a qualitative field validation.

During these tests, obstacle detection and tracking methodologies of circular obstacles were integrated [39]. Obstacles are detected using a LiDAR mounted on the vehicle. The desired velocity is computed by a goal-following navigation controller based on linear and angular proportional control, with-

⁶https://github.com/srmainwaring/asv_wave_sim/, accessed on July 25, 2025

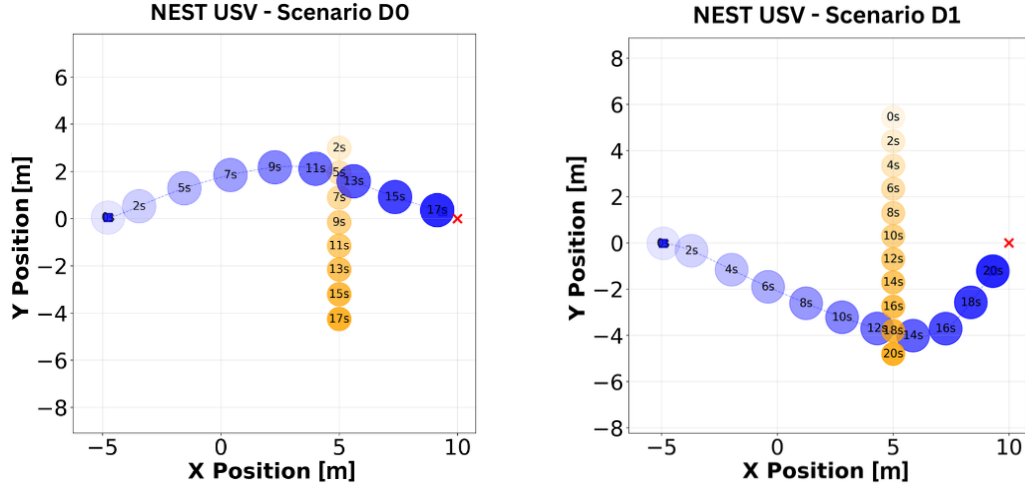


Figure 18: NEST Scenarios D0 (left) and D1 (right): XY trajectories in meters $[m]$. Robot trajectory in blue, obstacle trajectory in orange, and goal marked with a red cross. Markers along the paths indicate timestamps in seconds $[s]$.

Table 4: NEST Scenarios D0 and D1: Metrics

Metric	D0	D1
Path Length	14.81 m	16.26 m
Path Elongation	1.05×	1.14×
Min Clearance	2.19 m	1.19 m

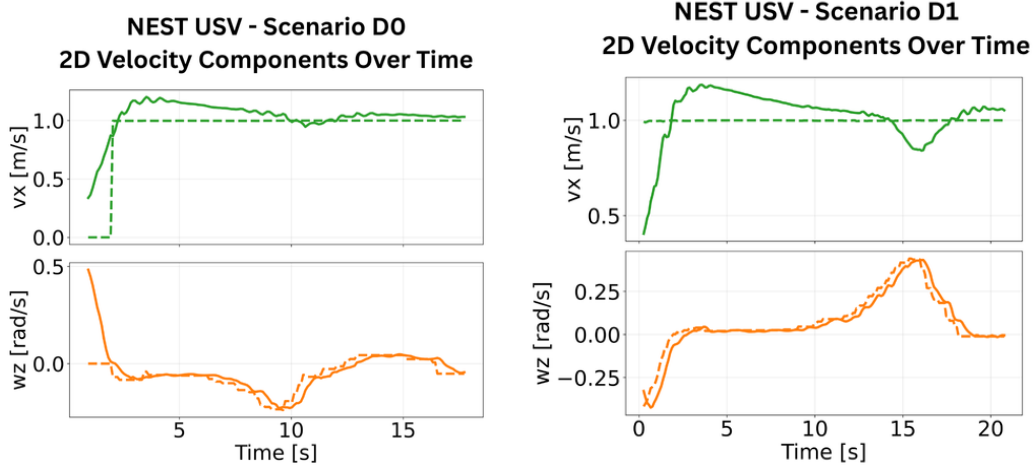


Figure 19: NEST Scenarios D0 (left) and D1 (right): Velocity components in $[m/s]$ and $[rad/s]$ over time $[s]$. Dotted lines represent commanded values; continuous lines represent actual values.

out considering obstacle information. Velocity feedback is provided by the vehicle’s IMU and GPS, and the low-level control that executes the command velocity relies on PID controllers for linear and angular velocities. The integration of these layers was done via ROS2 Jazzy running on the vehicle’s onboard computer.

Figure 20 shows the real-world environment from an aerial view and the environment representation with a static and a dynamic obstacle. In the first frame (first column), the desired path in green is represented, as well as the small buoy detection on the vehicles right. Then, S3VO commands a positive yaw angular velocity, leading to the second frame and describing an avoidance maneuver for the static obstacle. After aligning with the desired goal again, in the third frame the dynamic obstacle is detected. Considering the kayak’s velocity, the USV maintains heading but for a small correction. A video of this test and others can be seen in the link in the footnote ⁷, providing more insight into the described behaviour.

⁷<https://youtu.be/5Uaiglrl2zc>

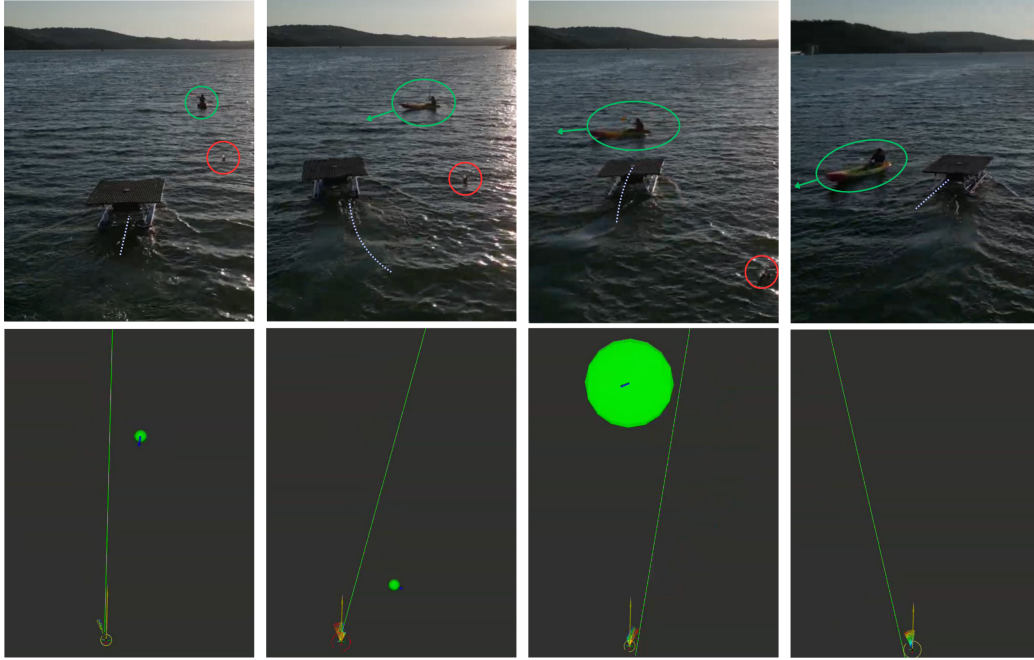


Figure 20: Real-World Testing in Barragem de Alqueva, Portugal. Small buoy in red is static, and kayak in green is a dynamic obstacle, moving in the green arrow's direction. The top row shows real-world images of the scenario, while the bottom row shows the environment representation from *RVIZ*, consisting of green obstacles with blue arrows showing their velocity. The agent navigation layer follows the path in green. The S3VO samples are displayed as a point cloud, where the cost goes from lowest in red to highest in purple. The agent first turns to the left to safely avoid the buoy, then passes the kayak by the right, considering it's velocity towards the left.

4.4. Computational Efficiency Analysis

The Heterogeneous Vehicle simulations were run on a laptop equipped with an Intel Core i7-12700H, 12th Gen, base 2.3 GHz), running Ubuntu 24.04 with ROS2 Jazzy, and Gazebo Harmonic.

The computational cost of the algorithm is primarily determined by the size of the velocity sample space that must be generated and evaluated at each planning step. The main obstacle-avoidance callback runs at 10 Hz and includes sample generation, pruning, and cost evaluation. To characterize this cost, the average processing time per callback for different sample-set sizes was measured. Table 5 summarizes the average time required to process 10, 100, 1000, and 10000 samples.

Table 5: Average processing time per callback for different sample sizes (single obstacle).

	10	100	1 000	10 000
Avg. Time per Callback [ms]	0.071	0.219	0.972	7.207

The number of obstacles also affects the computational cost. In the pruning stage, a larger number of obstacles tends to reduce the feasible velocity sample space, which can decrease the computational time. However, in the sample evaluation stage, each velocity sample is compared against all obstacles, increasing the computational cost. Table 6 presents the average processing time per callback for 1000 samples as the number of obstacles increases.

Table 6: Average processing time per callback for 1 000 samples with varying numbers of obstacles.

	1	5	10
Avg. Time per Callback [ms]	0.972	1.144	1.445

The results in Tables 5 and 6 show that the computational cost of the proposed method scales approximately linearly with both the number of samples and the number of obstacles. The processing time per callback remained well below 100 ms, corresponding to a 10 Hz planning rate, which is commonly used for real-time local planners, validating the real-time feasibility of S3VO. Although these measurements were obtained with a single-threaded sequential implementation, it should be noted that both the pruning and the evaluation of each velocity sample are independent operations, making the

algorithm well-suited to parallelization on multi-core CPUs or GPUs, mitigating the linear growth of computation time with the number of samples.

5. Conclusions

Uncrewed robotic vehicles operating in three-dimensional dynamic environments require obstacle avoidance approaches to ensure safe navigation. In maritime environments, 3D navigation is necessary not only for surface vehicles moving in 2D but interacting with 3D environments, but also for the full 3D navigation of aerial or underwater vehicles.

This work proposes S3VO, a unified obstacle avoidance algorithm for heterogeneous robotic platforms that builds upon the Velocity Obstacle approach in three-dimensional spaces. S3VO introduces two distinctive features: (i) a velocity space sampling framework generated from 6DOF kinematic constraints, making it easily adaptable to different vehicles, and (ii) velocity sample pruning and a risk-based cost function, assessing future collisions, collision risk, and whether the desired velocity is being followed.

The algorithm was implemented and validated through simulations of a holonomic surface vehicle, a differential-drive surface vehicle, and a multirotor aerial vehicle, demonstrating its adaptability to different platforms. It was compared with Artificial Potential Fields and Optimal Reciprocal Collision Avoidance. The results highlight several issues with ORCA when modeling complex scenarios, which lead to deadlocks and collisions, while S3VO was able to ensure safety in all of them. Real-world qualitative validation of this approach on the NEST ASV, integrating obstacle detection and tracking methodologies, was performed at Barragem de Alqueva, Portugal.

Future research should focus on the implementation of non-uniform sampling strategies and parallelized sampling implementations. Specifically, sampling density could be increased in critical regions of the velocity space, such as near the desired velocity command, the previous command, and the vehicle’s actuation limits. Additionally, the spherical obstacle and agent model should be enhanced in the future, and integrated with S3VO, allowing for more complex object representation models, such as ellipsoidal [44], cuboid, or superquadric [45], or even including obstacle semantics. Furthermore, more extensive and systematic validations with real-world vehicles should be conducted.

6. Acknowledgments

This work is partly funded by the European Union’s Horizon 2024 research and innovation programme under grant agreement No 101189723 (AERO-SUB).

References

- [1] G. G. O. Wind and E. I. London, *G+ Global Offshore Wind Health & Safety Organisation – 2023 Incident Data Report*, G+ Global Offshore Wind Health & Safety Organisation, Technical Report, 2023. Accessed: Dec. 12, 2024. [Online]. Available: <https://www.gplusoffshorewind.com/?a=1538335>.
- [2] Daniel F. Campos, Eduardo P. Gonçalves, Hugo J. Campos, Maria I. Pereira, and Andry M. Pinto, *Nautilus: An autonomous surface vehicle with a multilayer software architecture for offshore inspection*, *Journal of Field Robotics*, vol. 41, no. 4, pp. 966–990, Wiley, 2024. doi: 10.1002/rob.22267.
- [3] Pedro Pereira, Renato Silva, João Marques, Raul Campilho, Aníbal Matos, and Andry Pinto, *Raya: A Bio-Inspired AUV for Inspection and Intervention of Underwater Structures*, *IEEE Access*, IEEE, 2025. doi: 10.1109/ACCESS.2025.xxxxxxx.
- [4] Rafael M. Claro, Maria I. Pereira, Francisco S. Neves, and Andry M. Pinto, *Energy Efficient Path Planning for 3D Aerial Inspections*, *IEEE Access*, vol. 11, pp. 32152–32166, IEEE, 2023. doi: 10.1109/ACCESS.2023.3262837.
- [5] Paolo Fiorini and Zvi Shiller, *Motion planning in dynamic environments using the relative velocity paradigm*, In Proc. IEEE Int. Conf. Robotics and Automation, Atlanta, GA, USA, May 1993, pp. 560–565. doi: 10.1109/ROBOT.1993.292038.
- [6] S. D. Pendleton, H. Andersen, X. Du, X. Shen, M. Meghjani, Y. H. Eng, D. Rus, and M. H. Ang, *Perception, planning, control, and coordination for autonomous vehicles*, *Machines*, vol. 5, no. 1, p. 6, Mar. 2017. doi: <https://doi.org/10.3390/machines5010006>

- [7] Xu Wang, Xiaofei Yang, Yucheng Zheng, Jianzhen Li, Shihong Ding, Zhengrong Xiang, and Bin Zhang, *A Novel Formation Control Strategy for USVs With Improved DDPG: Simulation and Field Test*, *IEEE Internet of Things Journal*, pp. 1–1, IEEE, 2025. doi: 10.1109/JIOT.2025.3597968.
- [8] O. Khatib, *Real-time obstacle avoidance for manipulators and mobile robots*, *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986. doi: 10.1177/027836498600500106.
- [9] J.-F. Duhé, S. Victor, and P. Melchior, *Contributions on artificial potential field method for effective obstacle avoidance*, *Fractional Calculus and Applied Analysis*, vol. 24, pp. 421–446, 2021. doi: 10.1515/fca-2021-0019.
- [10] D. Fox, W. Burgard, and S. Thrun, *The dynamic window approach to collision avoidance*, *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.
- [11] M. Missura and M. Bennewitz, *Predictive collision avoidance for the dynamic window approach*, *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, Montreal, QC, Canada, 2019, pp. 8620–8626. doi: 10.1109/ICRA.2019.8794386.
- [12] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert, *Constrained model predictive control: Stability and optimality*, *Automatica*, vol. 36, no. 6, pp. 789–814, 2000.
- [13] C. Liu, S. Lee, S. Varnhagen, and H. E. Tseng, *Path planning for autonomous vehicles using model predictive control*, *Proc. IEEE Intelligent Vehicles Symposium (IV)*, Los Angeles, CA, USA, 2017, pp. 174–179. doi: 10.1109/IVS.2017.7995716.
- [14] Seungtaek Kim, Kyoungseok Han, and Seibum B. Choi, *Imitation Learning of Nonlinear Model Predictive Control for Emergency Collision Avoidance*, *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 1, pp. 2908–2922, 2024. doi: 10.1109/TIV.2023.3309962.
- [15] S. Quinlan and O. Khatib, *Elastic bands: Connecting path planning and control*, *Proc. IEEE Int. Conf. Robotics and Automation*, Atlanta, GA, USA, 1993, pp. 802–807. doi: 10.1109/ROBOT.1993.291936.

- [16] C. Rösmann, W. Feiten, T. Wösch, F. Hoffmann, and T. Bertram, *Trajectory modification considering dynamic constraints of autonomous robots*, Proc. 7th German Conf. Robotics (ROBOTIK), Munich, Germany, 2012, pp. 1–6.
- [17] L. A. Nguyen, T. D. Pham, T. D. Ngo, and X. T. Truong, *A proactive trajectory planning algorithm for autonomous mobile robots in dynamic social environments*, Proc. Int. Conf. Ubiquitous Robots (UR), pp. 309–314, 2020.
- [18] K. Almazrouei, I. Kamel, and T. Rabie, *Dynamic obstacle avoidance and path planning through reinforcement learning*, Applied Sciences, vol. 13, no. 14, p. 8174, 2023.
- [19] G. Zhang, L. Zhihao, J. Li, Y. Shu, and X. Zhang, “Reinforcement learning-driven autonomous navigation strategy for rotor-assisted vehicles via integral event-triggered mechanism,” *Transportation Research Part D: Transport and Environment*, vol. 146, p. 104841, 2025.
- [20] M. I. Pereira and A. M. Pinto, *Reinforcement learning-based robot navigation using illegal actions for autonomous docking of surface vehicles in unknown environments*, *Engineering Applications of Artificial Intelligence*, vol. 133, p. 108506, 2024. doi: 10.1016/j.engappai.2024.108506.
- [21] F. S. Neves, L. M. Branco, M. I. Pereira, R. M. Claro, and A. M. Pinto, *A multimodal learning-based approach for autonomous landing of UAV*, Proc. IEEE/ASME Int. Conf. Mechatronic and Embedded Systems and Applications (MESA), Genova, Italy, 2024, pp. 1–8. doi: 10.1109/MESA61532.2024.10704866.
- [22] J. Li, G. Zhang, C. Liu, and W. Zhang, “COLREGs-constrained adaptive fuzzy event-triggered control for underactuated surface vessels with the actuator failures,” *IEEE Transactions on Fuzzy Systems*, vol. 29, no. 12, pp. 3822–3832, 2020.
- [23] Paolo Fiorini and Zvi Shiller, *Motion planning in dynamic environments using velocity obstacles*, *The International Journal of Robotics Research*, vol. 17, no. 7, pp. 760–772, 1998. doi: 10.1177/027836499801700706.
- [24] Jur van den Berg, Ming Lin, and Dinesh Manocha, *Reciprocal velocity obstacles for real-time multi-agent navigation*, Proc. IEEE Int. Conf.

- Robotics and Automation (ICRA), Pasadena, CA, USA, 2008, pp. 1928–1935. doi: 10.1109/ROBOT.2008.4543489.
- [25] Jur van den Berg, Stephen J. Guy, Ming Lin, and Dinesh Manocha, *Reciprocal n-body collision avoidance*, In *Robotics Research: ISRR 2011*, Springer, pp. 3–19. doi: 10.1007/978-3-642-19457-3₁.
 - [26] David Wilkie, Jur van den Berg, and Dinesh Manocha, *Generalized velocity obstacles*, Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS), St. Louis, MO, USA, 2009, pp. 5573–5578. doi: 10.1109/IROS.2009.5354175.
 - [27] Jamie Snape, Jur van den Berg, Stephen J. Guy, and Dinesh Manocha, *The hybrid reciprocal velocity obstacle*, *IEEE Transactions on Robotics*, vol. 27, no. 4, pp. 696–706, Aug. 2011. doi: 10.1109/TRO.2011.2120810.
 - [28] Javier Alonso-Mora, Andreas Breitenmoser, Martin Rufli, Paul Beardsley, and Roland Siegwart, *Optimal reciprocal collision avoidance for multiple non-holonomic robots*, In *Distributed Autonomous Robotic Systems*, Springer, 2013, pp. 203–216. doi: 10.1007/978-3-642-32723-0₁₅.
 - [29] Boris Kluge and Erwin Prassler, *Recursive probabilistic velocity obstacles for reflective navigation*, In *Field and Service Robotics*, Springer, 2006, pp. 71–79. doi: 10.1007/10991459₈.
 - [30] Daniel Filipe Campos, Aníbal Matos, and Andry Maykol Pinto, *An adaptive velocity obstacle avoidance algorithm for autonomous surface vehicles*, Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS), Macau, China, 2019, pp. 8089–8096. doi: 10.1109/IROS40897.2019.8968156.
 - [31] Zhanteng Xie and Philip Dames, *DRL-VO: Learning to Navigate Through Crowded Dynamic Scenes Using Velocity Obstacles*, *IEEE Transactions on Robotics*, vol. 39, no. 4, pp. 2700–2719, 2023. doi: 10.1109/TRO.2023.3257549.
 - [32] L. Bonanni, D. Meli, A. Castellini, and A. Farinelli, “Monte Carlo Tree Search with Velocity Obstacles for safe and efficient motion planning in dynamic environments,” in *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2025)*, Detroit, Michigan, USA, May 19–23, 2025.

- [33] Xiaofei Yang, Mengmeng Lou, Jiabao Hu, Hui Ye, Zhiyu Zhu, Hao Shen, Zhengrong Xiang, and Bin Zhang, *A human-like collision avoidance method for USVs based on deep reinforcement learning and velocity obstacle*, *Expert Systems with Applications*, vol. 254, p. 124388, 2024. doi: 10.1016/j.eswa.2024.124388.
- [34] Thomas Battisti and Riccardo Muradore, *A velocity obstacles approach for cooperative landing of a teleoperated aerial robot on a moving platform*, *Autonomous Robots*, vol. 44, pp. 153–170, 2020. doi: 10.1007/s10514-019-09887-8.
- [35] Yazdi I. Jenie, Erik-Jan van Kampen, Cornelis C. de Visser, Joost Ellerbroek, and Jacco M. Hoekstra, *Three-dimensional velocity obstacle method for uncoordinated avoidance maneuvers of unmanned aerial vehicles*, *Journal of Guidance, Control, and Dynamics*, vol. 39, no. 10, pp. 2312–2323, 2016. doi: 10.2514/1.G001715.
- [36] Myungwhan Choi, Areeya Rubenecia, Taeshik Shon, and Hyo Hyun Choi, *Velocity obstacle based 3D collision avoidance scheme for low-cost micro UAVs*, *Sustainability*, vol. 9, no. 7, p. 1174, 2017. doi: 10.3390/su9071174.
- [37] José Martins, Pedro Pereira, Raul Campilho, and Andry Pinto, *Wave-motion compensation for USV–UAV cooperation: A model predictive controller approach*, In *Proceedings of the 2024 20th IEEE/ASME International Conference on Mechatronic and Embedded Systems and Applications (MESA)*, IEEE, 2024, pp. 1–8.
- [38] Pedro Pereira, Raul Campilho, and Andry Pinto, *An inverse kinematics approach for the analysis and active control of a four-UPR motion-compensated platform for UAV–ASV cooperation*, *Machines*, vol. 11, no. 4, p. 478, 2023. doi: 10.3390/machines11040478.
- [39] Mateusz Przybyła, *Detection and tracking of 2D geometric obstacles from LRF data*, *Proc. IEEE Int. Workshop on Robot Motion and Control (RoMoCo)*, Wasowo Palace, Poland, 2017, pp. 135–141. doi: 10.1109/RoMoCo.2017.8003904.
- [40] Chee Yong Tan, Sunan Huang, Kok Kiong Tan, and Rodney Swee Huat Teo, *Three Dimensional Collision Avoidance for Multi Unmanned Aerial*

- Vehicles Using Velocity Obstacle*, J. Intell. Robot. Syst., vol. 97, no. 1, pp. 227–248, 2020. doi: 10.1007/s10846-019-01055-5.
- [41] Yong Liao, Yuxin Wu, Shichang Zhao, and Dan Zhang, *Unmanned aerial vehicle obstacle avoidance based custom elliptic domain*, Drones, vol. 8, no. 8, p. 397, 2024. doi: 10.3390/drones8080397.
 - [42] Federico Vesentini, Nicola Piccinelli, and Riccardo Muradore, *Velocity obstacle-based trajectory planner for anthropomorphic arms*, Eur. J. Control, vol. 75, p. 100901, 2024. doi: 10.1016/j.ejcon.2023.100901.
 - [43] J. Karwowski and W. Szykiewicz, “Quantitative metrics for benchmarking human-aware robot navigation,” *IEEE Access*, vol. 11, pp. 79941–79953, 2023.
 - [44] B. H. Lee, J. D. Jeon, and J. H. Oh, “Velocity obstacle based local collision avoidance for a holonomic elliptic robot,” *Autonomous Robots*, vol. 41, no. 6, pp. 1347–1363, 2017.
 - [45] M. Ginesi, D. Meli, A. Roberti, N. Sansonetto, and P. Fiorini, “Dynamic movement primitives: Volumetric obstacle avoidance using dynamic potential functions,” *Journal of Intelligent & Robotic Systems*, vol. 101, no. 4, p. 79, 2021.